

Agent Prompts & Developer Guide

Regulatory Governance & Action Platform — Pre-Build Package

Four agents that can be built and tested before the detailed use case arrives.

1. How to Use This Document

This covers the four agents whose structure will not change when the detailed use case arrives. They can be built, tested, and tuned now. Each section gives the agent's purpose, its input, its output schema, the prompt to start from, and how to test it.

Two rules that make Thursday cheap

Rule 1 — Output schemas are frozen.

Once agreed, do not change the field names or types in any output schema. Prompts will be re-tuned on Thursday; schemas must not move, because everything downstream depends on them. If a schema genuinely must change, it is a team decision, not an individual one.

Rule 2 — No domain vocabulary inside prompt text.

Every industry-specific term — scope dimensions, product names, plant names, regulator names, taxonomy labels — must be injected from a config object at call time, never hard-coded into the prompt string. Thursday's changes should be edits to a JSON file, not rewrites of four prompts under time pressure.

Common call pattern

agent(input_object, config) → structured JSON matching a fixed schema

// Every agent follows this shape. Config carries all domain vocabulary.

// Prompts reference config values as variables, never as literals.

Shared implementation requirements

Request JSON only. State explicitly in the system prompt: respond with valid JSON and nothing else — no preamble, no explanation, no markdown code fences.

Strip fences defensively before parsing anyway. Models add them despite instructions.

Wrap every parse in try/catch. On parse failure, retry once, then fail loudly — never silently return a default.

Log the model name and prompt version with every call. This is a Responsible AI requirement (FR-53), not optional polish.

Set temperature low (0–0.2). These are classification and extraction tasks, not creative ones.

Every agent that produces a recommendation must also produce a confidence value. No exceptions.

2. Agent 1 — Triage

Maps to FR-06 through FR-09. Runs immediately after Acquisition, before anything else.

Decides whether a retrieved document deserves attention at all, and what kind of thing it is. This is the gate that prevents irrelevant documents from consuming any downstream processing.

Input

```
{
  document: { // from Acquisition layer
    doc_id, source, title, doc_type,
    published_date, source_url, raw_text
  },
```

```
config: {  
  organization_profile, // free text description of the org  
  scope_dimensions, // e.g. product lines, plants, markets  
  tracked_regulations // list of {id, title, identifier}  
}
```

```
}
```

Output schema — FROZEN

```
{  
  "doc_id": "string",  
  "in_scope": true | false,  
  "scope_reasoning": "string", // always populated, even when false  
  "relation_to_existing": "new" | "amendment" | "duplicate" | "unrelated",  
  "related_regulation_id": "string | null",  
  "confidence": 0.0-1.0,  
  "urgency_flag": "routine" | "elevated" | "urgent",  
  "routing_decision": "queue_for_decomposition" | "discard" | "needs_human_triage"  
}
```

Prompt

SYSTEM:

You are a regulatory triage analyst. You assess whether a regulatory document is relevant to a specific organization, and classify its relationship to regulations the organization already tracks.

You do not make compliance decisions. You route documents for review.

Respond with valid JSON only. No preamble, no explanation outside the JSON, no markdown code fences.

USER:

ORGANIZATION PROFILE:

{{config.organization_profile}}

SCOPE DIMENSIONS:

{{config.scope_dimensions}}

REGULATIONS ALREADY TRACKED:

{{config.tracked_regulations}}

DOCUMENT TO TRIAGE:

Title: {{document.title}}

Source: {{document.source}}

Type: {{document.doc_type}}

Published: {{document.published_date}}

Text: {{document.raw_text}}

Assess the following:

1. IN SCOPE: Could this document plausibly create obligations for this organization, given its profile and scope dimensions? Err toward `in_scope=true` when uncertain — a false positive costs a human review, a false negative means a missed regulation.

2. RELATION: Compare against the tracked regulations list.

- "amendment" if it modifies a regulation already in that list
- "duplicate" if it restates something already tracked
- "new" if in scope but unrelated to anything tracked
- "unrelated" if not in scope

If amendment or duplicate, populate `related_regulation_id`.

3. CONFIDENCE: How certain are you of the relation classification?

Be conservative. Reserve values above 0.85 for cases where the document explicitly names the regulation it amends.

4. URGENCY: "urgent" only if an effective date or comment deadline appears to fall within 90 days. Otherwise "routine" or "elevated".

5. ROUTING:

- "discard" if `in_scope` is false
- "needs_human_triage" if confidence < 0.6, regardless of other fields
- otherwise "queue_for_decomposition"

Return JSON matching the required schema.

Testing

Feed it a genuinely irrelevant document (a food-safety notice) — should return `in_scope=false`, discard.

Feed it the July 2026 EPA NOx amendment NPRM with the 2023 rule in `tracked_regulations` — should return `relation=amendment` with the correct `related_regulation_id`.

Feed it a document with an ambiguous connection — confirm confidence drops below 0.6 and routing becomes `needs_human_triage`.

Confirm the 0.6 threshold is defined in ONE place in the codebase, shared with the divergence dashboard.

3. Agent 2 — Obligation Extraction

Maps to FR-11 through FR-13. The most domain-agnostic agent in the system — build this one first.

Breaks a regulation into discrete, individually trackable obligations. Every downstream stage operates on obligations, not on regulations, so the quality of this agent sets a ceiling on everything after it.

Input

```
{
  regulation: { id, title, identifier, raw_text, effective_date },
  config: { obligation_types } // allowed type values for this domain
```

```
}
```

Output schema — FROZEN

```
{
```

```
"regulation_id": "string",
```

```
"obligations": [
```

```
{
```

```
"clause_ref": "string", // e.g. "Article 8(2)", "§ 1036.104"
```

```
"summary": "string", // plain language, one sentence
```

```
"source_quote": "string", // VERBATIM from source text
```

```
"obligation_type": "string", // from config.obligation_types
```

```
"effective_date": "YYYY-MM-DD | null",
```

```
"extraction_confidence": 0.0-1.0
```

```
}
```

```
],
```

```
"extraction_notes": "string" // ambiguities flagged for SME review
```

```
}
```

Prompt

SYSTEM:

You are a regulatory analyst who decomposes regulations into discrete, individually trackable obligations.

An obligation is a single requirement that one person could be made accountable for. If a clause contains two separable requirements, emit two obligations. If a clause is definitional, procedural background, or scope-setting with no requirement, do not emit an obligation for it.

Every obligation MUST include a verbatim source_quote copied exactly from the supplied text. Never paraphrase into source_quote. If you cannot locate exact supporting text, do not emit that obligation.

Respond with valid JSON only. No preamble, no markdown code fences.

USER:

ALLOWED OBLIGATION TYPES:

{{config.obligation_types}}

REGULATION:

Title: {{regulation.title}}

Identifier: {{regulation.identifier}}

Parent effective date: {{regulation.effective_date}}

TEXT:

{{regulation.raw_text}}

Extract every discrete obligation. For each:

- `clause_ref`: the article, section, or paragraph number as written in the source. If genuinely unnumbered, describe the location instead.
- `summary`: one sentence, plain language, stating what must be done and by whom. No legal jargon.
- `source_quote`: verbatim text supporting the obligation. Keep it short — the specific requirement sentence, not the whole clause.
- `obligation_type`: choose from the allowed types above. If none fit, use the closest and note it in `extraction_notes`.
- `effective_date`: only if this obligation has its own date distinct from the parent regulation. Otherwise null.
- `extraction_confidence`: lower this when the clause is ambiguous, uses conditional language, or cross-references text not supplied.

In `extraction_notes`, flag anything an expert should review: ambiguous scope, unresolved cross-references, conditional applicability, or clauses you deliberately skipped and why.

Return JSON matching the required schema.

Testing

Run against the EPA MY2027 NOx rule text. Expect distinct obligations for the NOx limit, PM limit, warranty terms, and useful-life provisions — not one merged blob.

Verify every `source_quote` appears verbatim in the input. Write an automated check for this; it is the foundation of the citation constraint.

Run against PFAS restriction text — confirms it works on a completely different regulatory style and regulator.

Run the same regulation twice and compare. Extraction should be broadly stable; wild variance means temperature is too high.

4. Agent 3 — Amendment Diff

Maps to FR-46. Your amendment-flow differentiator. Test against the real July 2026 EPA NPRM.

Compares two versions of a regulation at obligation level and classifies what changed. The "removed or relaxed" category is the one most competing approaches miss — it lets the platform tell an organization it can stand work down, not only pick work up.

Input

```
{
  prior_version: { regulation_id, version_label, obligations: [...] },
  new_version: { regulation_id, version_label, obligations: [...] }
}
```

// Obligations in both are the output of Agent 2.

Output schema — FROZEN

```
{
  "regulation_id": "string",
  "comparison": [
```

```
{
  "prior_obligation_ref": "string | null", // null when status=new
  "new_obligation_ref": "string | null", // null when status=removed
  "status": "unchanged" | "modified" | "new" | "removed_or_relaxed",
  "change_summary": "string", // empty when unchanged
  "direction": "stricter" | "relaxed" | "lateral" | "n/a",
  "confidence": 0.0-1.0
}
],
"overall_summary": "string"
}
```

Prompt

SYSTEM:

You compare two versions of a regulation and classify what changed at the level of individual obligations.

You are not assessing whether changes are good or bad. You are establishing, precisely, what is different — so that downstream work already underway can be re-validated, extended, or stood down.

Match obligations across versions by substance, not by clause number.

Clause numbering frequently shifts between versions while the underlying requirement stays the same.

Respond with valid JSON only. No preamble, no markdown code fences.

USER:

PRIOR VERSION ({{prior_version.version_label}}):

{{prior_version.obligations}}

NEW VERSION ({{new_version.version_label}}):

{{new_version.obligations}}

Produce one comparison entry per obligation relationship:

- "unchanged": substance is identical. Renumbering or pure wording changes with no substantive effect are still unchanged.
- "modified": the requirement still exists but its substance changed — a threshold, a date, a scope, a condition.
- "new": present in the new version with no counterpart in the prior.
- "removed_or_relaxed": present in the prior version but removed, or materially loosened. Treat deferred deadlines and reduced requirements as relaxed, not modified.

For every non-unchanged entry, set direction:

- "stricter": more burden, tighter limit, earlier deadline

- "relaxed": less burden, looser limit, later deadline
- "lateral": changed but not clearly more or less burdensome
- "n/a": only for unchanged

change_summary must state concretely what changed — name the old value and the new value where both are known. Do not write "requirements were updated".

Lower confidence when matching across versions is uncertain, or when a change's direction is genuinely arguable.

Return JSON matching the required schema.

Testing

Primary test case: the 2023 EPA heavy-duty NOx rule vs the July 2026 NPRM revising it. Expect the warranty provision as modified/relaxed and the useful-life increase as relaxed (deferred to MY2030), with the core NOx limit unchanged.

Feed identical inputs both sides — every entry must come back unchanged. A false "modified" here means the prompt is over-eager.

Renumber the clauses in one version without changing substance — should still return unchanged, proving substance matching works.

Confirm at least one removed_or_relaxed result exists in your demo path; it is the category that carries the differentiator.

5. Agent 4 — Verification

Differentiator #1. Runs against another agent's output before a human ever sees it.

Independently challenges a recommendation produced by another agent. This is not the same call restated — it is a deliberately adversarial second pass asked to find the strongest reason the first answer is wrong. Its output becomes an additional signal on the recommendation, not a replacement for it.

Input

```
{
  recommendation: {
    agent_name, decision, rationale,
    confidence, citation, source_context
  }
}
```

// Deliberately domain-blind: it critiques reasoning, not subject matter.

Output schema — FROZEN

```
{
  "verdict": "supported" | "questionable" | "unsupported",
  "strongest_counterargument": "string",
  "unsupported_claims": ["string"], // assertions not backed by cited text
  "missing_considerations": ["string"],
  "revised_confidence_suggestion": 0.0-1.0,
  "verification_confidence": 0.0-1.0
}
```

}

Prompt

SYSTEM:

You are a reviewer whose job is to find weaknesses in an analytical recommendation before a human acts on it.

You are not asked to agree. You are asked to construct the strongest honest case that the recommendation is wrong, incomplete, or insufficiently supported by its cited evidence.

If the recommendation is genuinely sound, say so — do not manufacture objections. A reviewer who always objects is as useless as one who never does.

Respond with valid JSON only. No preamble, no markdown code fences.

USER:

RECOMMENDATION UNDER REVIEW

Produced by: {{recommendation.agent_name}}

Decision: {{recommendation.decision}}

Stated confidence: {{recommendation.confidence}}

Rationale: {{recommendation.rationale}}

Citation: {{recommendation.citation}}

SOURCE TEXT THE RECOMMENDATION RELIES ON:

{{recommendation.source_context}}

Assess:

1. Is every factual assertion in the rationale actually supported by the source text supplied? List any that are not in unsupported_claims. Assertions that sound plausible but are not in the source belong in this list.
2. What is the strongest honest argument that this decision is wrong? State it in strongest_counterargument. If there is genuinely no strong counterargument, say so explicitly rather than inventing one.
3. What relevant considerations does the rationale not address?
4. Verdict:
 - "supported": reasoning follows from cited evidence, no material gaps
 - "questionable": defensible but with real gaps or unsupported steps
 - "unsupported": the cited evidence does not sustain the decision
5. revised_confidence_suggestion: what the original confidence should be, in your assessment. May be higher or lower than stated.

Return JSON matching the required schema.

Testing

Feed it a deliberately weak recommendation with a rationale asserting facts absent from the source — should populate `unsupported_claims` and return questionable or unsupported.

Feed it a genuinely strong, well-cited recommendation — should return supported without inventing objections. This is the harder test and the one most likely to fail early.

Confirm it works against output from BOTH the Triage and Obligation Extraction agents, proving it is genuinely agent-agnostic.

Decide as a team how verdict affects routing — recommended: "unsupported" forces human review regardless of the original confidence value.

6. Next Steps

Build order

Build in this sequence — each one de-risks the next.

Definition of done, per agent

Returns valid JSON on 10 consecutive calls without a parse failure.

Output validates against the frozen schema — write the validator, do not eyeball it.

Tested against at least two genuinely different source documents.

Prompt stored as a versioned text file, not inline in application code.

All domain vocabulary confirmed to come from config, with zero hard-coded industry terms in the prompt.

Model name and prompt version logged with every call.

Two-day sequencing

Day 1

Agree and freeze all four output schemas as a team — do this first, before any prompt work.

Build and test Obligation Extraction against EPA NOx and PFAS text.

Start Amendment Diff once extraction produces usable output.

Day 2

Finish Amendment Diff; confirm the `removed_or_relaxed` case works on the real EPA amendment.

Build Triage once Dev 4's seed data schema is available.

Build Verification against real outputs from the first three.

Save a fixture file of known-good input/output pairs for every agent — this is your regression check on Thursday.

What is deliberately NOT in this document

The Applicability and Impact Fan-Out agents are excluded because their prompts depend on the scope model and impact taxonomy, which the detailed use case may change. Their output schemas can be agreed now; their prompts should wait. The Dedup and Overlap agent sits between the two groups — build it if the four above are finished early.

Thursday handover checklist

Four working agents with fixture files proving they work.

A written note per agent on what config values it expects.

A list of every place a domain term would need to change if the industry changed — there should be very few, and all in config.

Known failure modes per agent, written down, so nobody rediscovers them under time pressure.